

MODUL XI

TEMA

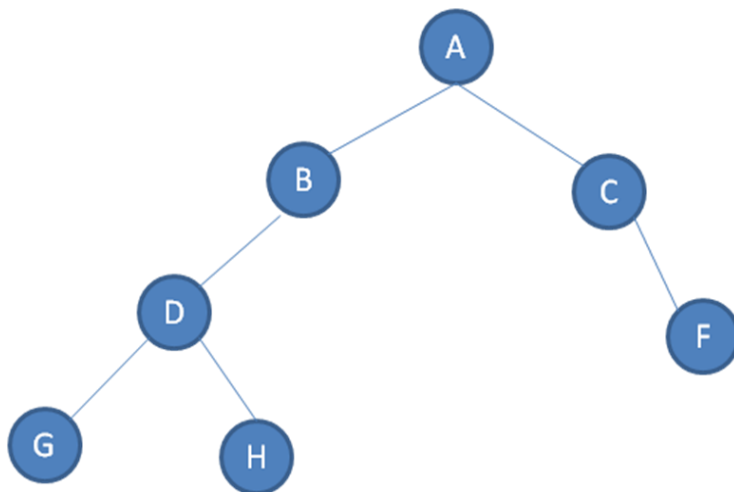
Tree/Pohon.

TUJUAN

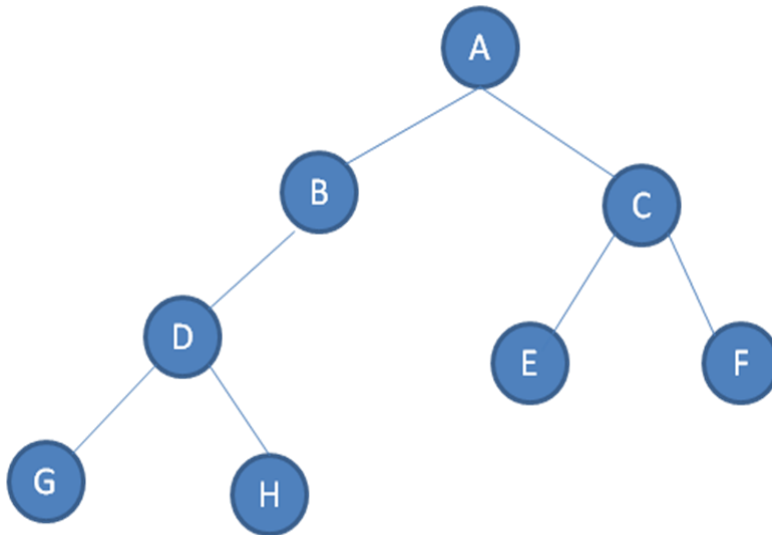
Agar mahasiswa dapat mengetahui, memahami dan menggunakan konsep Tree/Pohon untuk menyelesaikan permasalahan dalam kehidupan sehari-hari.

MATERI

TREE adalah kumpulan dari NODE yang saling terhubung yang disusun secara hirarki menyerupai struktur sebuah pohon yang terbalik.

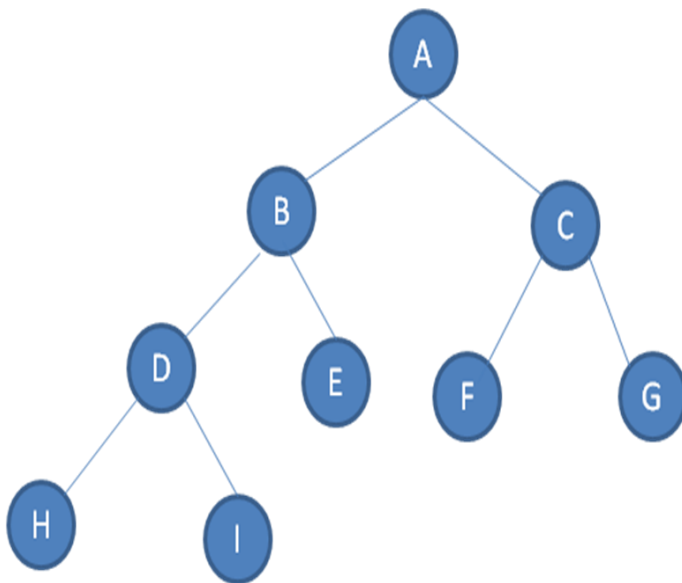


Node teratas dari tree disebut dengan node root. Node root adalah node yang pertama kali di implementasikan dan dapat memiliki anak berupa node node di tingkat hirarki dibawahnya ada disebut dengan “sub tree”.



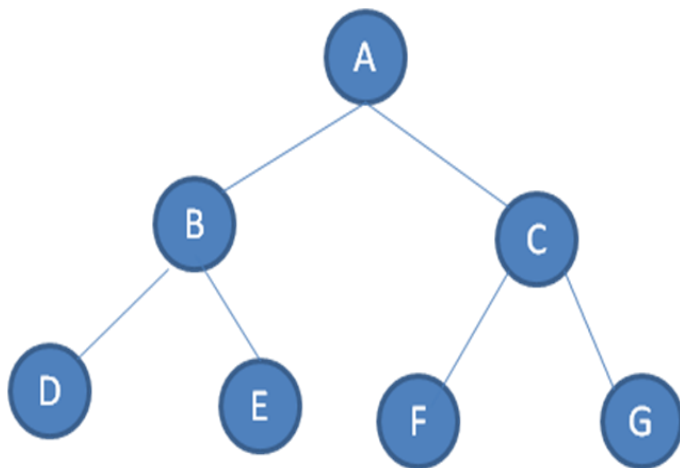
BINARY TREE

Salah satu jenis dari TREE adalah binary tree. Ciri khas dari binary tree adalah setiap node hanya boleh memiliki maksimal dua child.

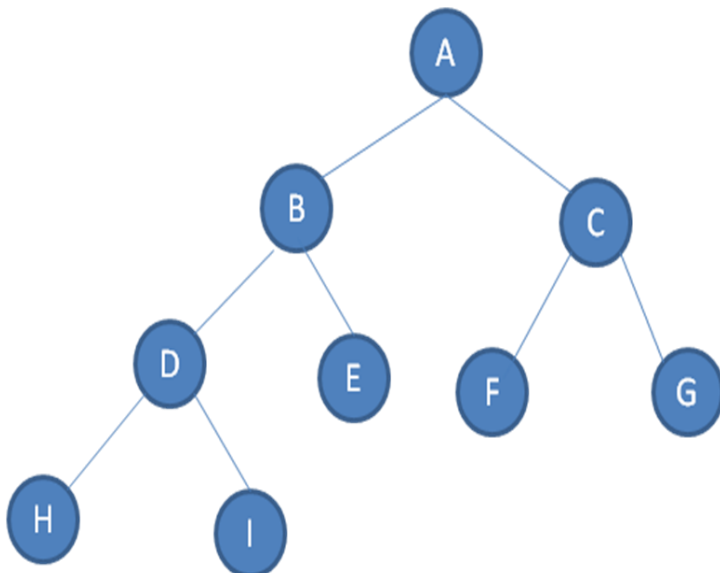


MACAM-MACAM BINARY TREE

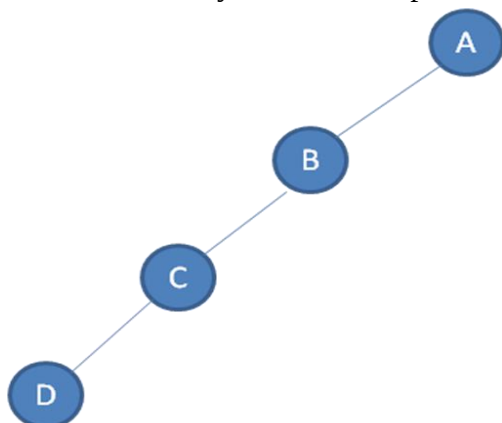
FULL Binary Tree : Setiap node (kecuali leaf) memiliki dua anak, dan setiap subtree memiliki panjang path yang sama.



COMPLETE Binary Tree : tiap node kecuali leaf memiliki dua child dan untuk panjang path dari sub tree berbeda.



SKEWED Binary TREE : Setiap node kecuali leaf hanya memiliki satu child.



ATURAN JUMLAH NODE

Jumlah maksimum node pada setiap level dari Binary TREE dapat dirumuskan sbb : 2^n

Jumlah Node max perlevel

Level (depth)

2^0

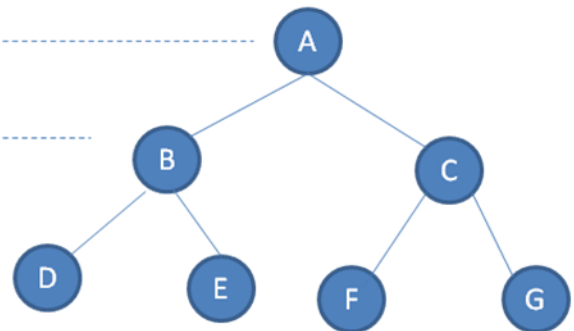
0

2^1

1

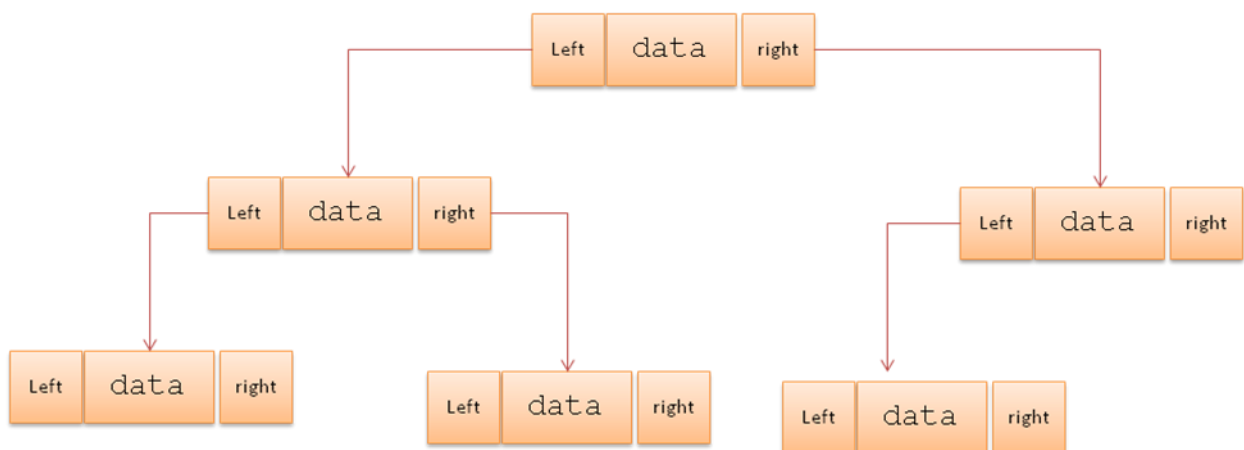
2^2

2



IMPLEMENTASI TREE

Untuk mengimplementasikan Tree pada program digunakan Double Linked List Non Circular secara Rekursif.



DEKLARASI TREE

```
class Tree {  
    public $data;  
    public $left;  
    public $right;
```

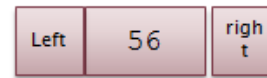
```
class BinaryTree {  
    public $root;
```

MENAMBAHKAN DATA PADA TREE

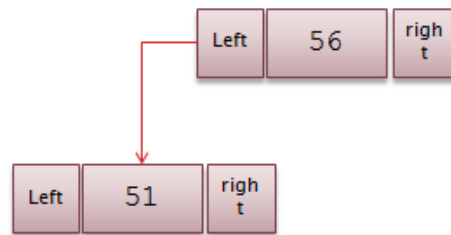
Pada add data di Tree, data yang pertama kali di tambahkan akan menjadi **root**, selanjutnya saat kita menambahkan **child**nya, apabila data lebih kecil dari data di ROOT akan ditempatkan di bagian kiri, sebaliknya apabila data lebih besar di tempatkan disebelah kanan.

ilustrasi :

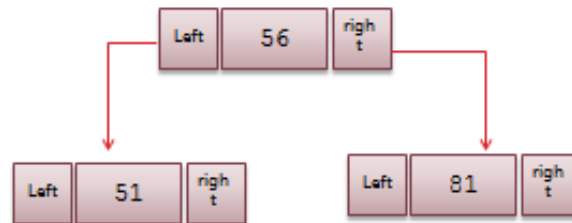
1. Add(&root, 56)



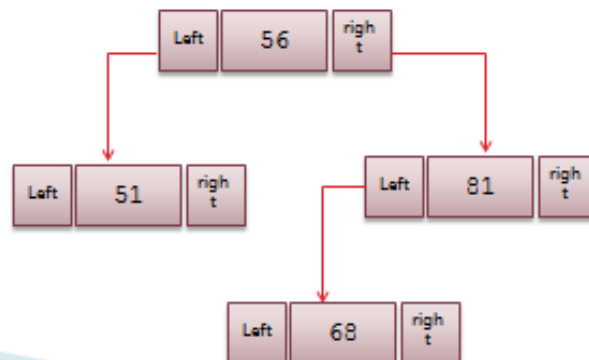
2. Add(&root, 51)



3. Add(&root, 81)



4. Add(&root, 68)



```

public function insert($d) {
    $newNode = new Tree($d);
    if ($this->root == null) {
        $this->root = $newNode;
    } else {
        $this->insertNode($this->root, $newNode);
    }
}

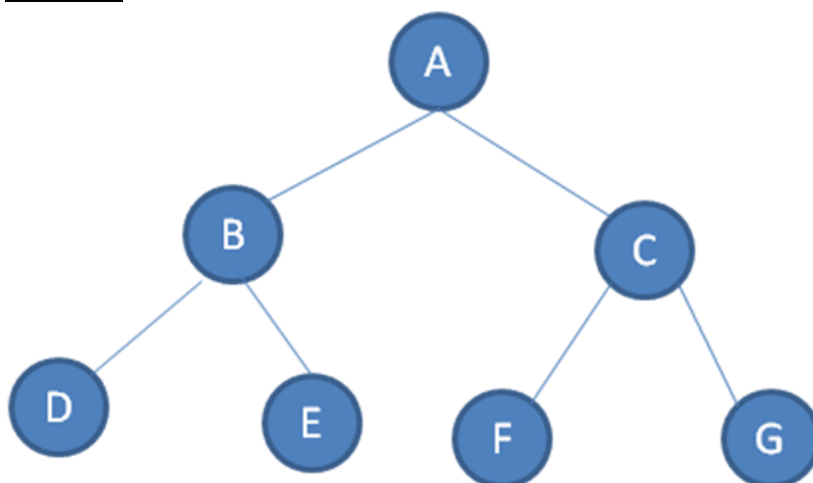
private function insertNode(&$node, &$newNode) {
    if ($node == null) {
        $node = $newNode;
    } else {
        if ($newNode->data < $node->data) {
            if ($node->left == null) {
                $node->left = $newNode;
            } else {
                $this->insertNode($node->right, $newNode);
            }
        } else {
            if ($node->right == null)
            {
                $node->right = $newNode;
            }
            else
            {
                $this->insertNode($node->right, $newNode);
            }
        }
    }
}
}

```

JENIS-JENIS TRAVERSE

- **PreOrder**: cetak node yang dikunjungi, kunjungi left, kunjungi right.
- **InOrder**: kunjungi left, cetak node yang dikunjungi, kunjungi right.
- **PostOrder**: kunjungi left, kunjungi right, cetak node yang dikunjungi.

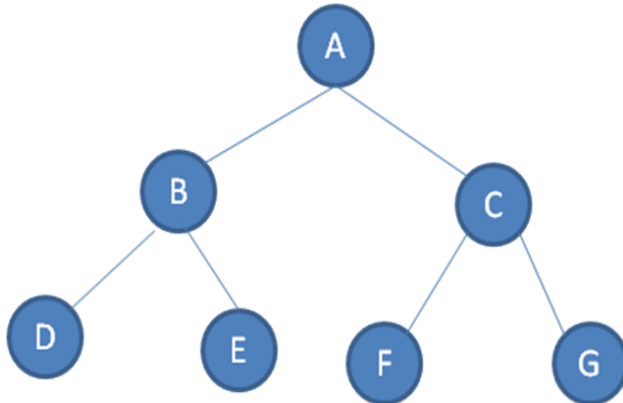
PreOrder



Hasil Output : A,B,D,E,C,F,G

```
public function preorder($node) {  
    if ($node !== null) {  
        echo $node->data . " ";  
        $this->preorder($node->left);  
        $this->preorder($node->right);  
    }  
}
```

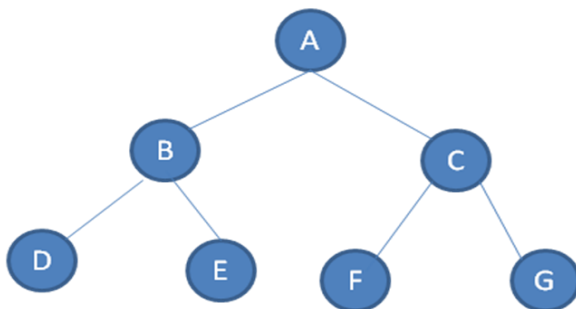
InOrder



Hasil Output : D,B,E,A,F,C,G

```
public function inorder($node) {  
    if ($node !== null) {  
        $this->inorder($node->left);  
        echo $node->data . " ";  
        $this->inorder($node->right);  
    }  
}
```

PostOrder



Hasil Output : D,E,B,F,G,C,A

```
public function postorder($node) {  
    if ($node !== null) {  
        $this->postorder($node->left);  
        $this->postorder($node->right);  
        echo $node->data . " ";  
    }  
}
```

PRAKTIKUM

1. Tree/pohon

```
class Tree {
    public $data;
    public $left;
    public $right;

    public function __construct($d)
    {
        $this->data = $d;
        $this->left = null;
        $this->right = null;
    }
}
```

```
class BinaryTree {
    public $root;

    public function __construct()
    {
        $this->root = null;
    }
}
```

```
public function insert($d) {
    $newNode = new Tree($d);
    if ($this->root === null) {
        $this->root = $newNode;
    } else {
        $this->insertNode($this->root, $newNode);
    }
}

private function insertNode(&$node, &$newNode) {
    if ($node == null) {
        $node = $newNode;
    } else {
        if ($newNode->data < $node->data) {
            if ($node->left == null) {
                $node->left = $newNode;
            } else {
                $this->insertNode($node->right, $newNode);
            }
        } else {
            if ($node->right == null)
            {
                $node->right = $newNode;
            }
            else
            {
                $this->insertNode($node->right, $newNode);
            }
        }
    }
}
```

```

public function inorder($node) {
    if ($node !== null) {
        $this->inorder($node->left);
        echo $node->data . " ";
        $this->inorder($node->right);
    }
}

public function preorder($node) {
    if ($node !== null) {
        echo $node->data . " ";
        $this->preorder($node->left);
        $this->preorder($node->right);
    }
}

public function postorder($node) {
    if ($node !== null) {
        $this->postorder($node->left);
        $this->postorder($node->right);
        echo $node->data . " ";
    }
}

```

```

public function cari($data)
{
    return $this->cariNode($this->root, $data);
}

```

```

public function tinggi($node)
{
    if ($node == null)
        return 0;
    else
    {
        $tinggi1 = $this->tinggi($node->left);
        $tinggi2 = $this->tinggi($node->right);
        if ($tinggi1 > $tinggi2)
            return $tinggi1 + 1;
        else
            return $tinggi2 + 1;
    }
}

```

```

public function daun($node)
{
    if ($node == null)
        echo "<br>Tree kosong";
    if ($node->left != null)
        $this->daun ($node->left);
    if ($node->right != null)
        $this->daun ($node->right);
    if ($node->left == null && $node->right == null)
        echo $node->data, " ";
}

// Contoh penggunaan
$str = new BinaryTree();
$str->insert(11);
$str->insert(55);
$str->insert(33);
$str->insert(8);
echo "Preorder : "; $str->preorder($str->root);
echo "<br>Inorder : "; $str->inorder($str->root);
echo "<br>Postorder : "; $str->postorder($str->root);
$str->cek(51);
$str->cek(33);
echo "<br>Jumlah Node : ", $str->hitung($str->root);
echo "<br>Kedalaman level : ", $str->tinggi($str->root);
echo "<br>Daun : ", $str->daun($str->root);

```

2. Tambahkan procedure berikut ini :
 - a. Fungsi cariNode dengan parameter node dan data, bersifat privat, dengan lagoritma sebagai berikut :
 1. Jika tidak terdapat ada node pada tree/null, maka nilai kembalian = 0/false;
 2. Jika data yang dicari sama dengan data yang terdapat pada node, maka nilai Kembali = 1/true;
 3. Jika data yang dicari lebih kecil dari data yang terdapat pada node, maka nilai Kembali berupa rekursif pemanggilan fungsi cariNode dimana data terdapat pada node bagian kiri/left;
 4. Jika tidak, maka dilakukan rekursif pemanggilan fungsi cariNode, dimana data yang dicari terdapat pada node bagian kanan/right.
 - b. Procedure cek dengan parameter data, bersifat public, dengan algoritma sebagai berikut :
 1. Jika cari data bernilai benar/true, maka data ditemukan;
 2. Jika tidak/false, maka data tidak ditemukan.
 - c. Fungsi hitung dengan parameter node, bersifat public, dengan algoritma sebagai berikut :
 1. Jika tidak terdapat node pada tree/sama dengan null/kosong, maka nilai Kembali = 0;
 2. Jika terdapat node pada tree, maka nilai Kembali = 1 + hitung (rekursif) sebelah kiri + hitung (rekursif) sebelah kanan.
3. Output program :


```

Preorder : 11 8 55 33
Inorder : 8 11 33 55
Postorder : 8 33 55 11
data 51 tidak ditemukan
data 33 ditemukan
Jumlah Node : 4
Kedalaman level : 3
Daun : 8 33
      
```

BUKU ACUAN

- Goodrich, Michael T.; Tamassia, Roberto; Mount, David. 2004. Data Structures and Algorithms in C++. WILEY.
- Hartono, Jogyanto. 1992. *Konsep Dasar Pemrograman Bahasa C*. Yogyakarta : Penerbit Andi
- Wahyudi, Bambang. 2004. "Pengantar Struktur Data & Algoritma". Yogyakarta: Penerbit Andi
- Yatini B., Indra; Nasution, Erliansya. 2005. "Algoritma & Struktur Data dengan C++". Graha Ilmu.